
Universidad de Las Americas (UDLA)
Facultad de Ingenieria y Negocios
Instituto de Matematicas, Fisica y Estadistica - Area de Estadistica

Examen Final: SQL

Estudiante: Pablo Villegas
Correo: pablo.pvill002@gmail.com
Dialecto: T-SQL (SQL Server)
Fecha: 15 de junio de 2026

Problema 1 - SQL: JOIN y agregaciones

Se consideran las tablas CLIENTES (ID, NOMBRE, CIUDAD) y VENTAS (ID_VENTA, ID_CLIENTE, MONTO), con los datos del enunciado. A continuacion se resuelve cada apartado con su codigo SQL y el resultado esperado.

a) Mostrar nombre, ciudad y monto usando JOIN

```
SELECT C.NOMBRE, C.CIUDAD, V.MONTO
FROM   CLIENTES C
INNER JOIN VENTAS V ON C.ID = V.ID_CLIENTE;
```

El INNER JOIN une cada venta con su cliente a traves de la clave C.ID = V.ID_CLIENTE.

NOMBRE	CIUDAD	MONTO
Juan	Concepcion	100000
Juan	Concepcion	150000
Maria	Los Angeles	50000
Pedro	Concepcion	200000

b) Total vendido por cliente

```
SELECT C.NOMBRE, SUM(V.MONTO) AS TOTAL_VENDIDO
FROM   CLIENTES C
INNER JOIN VENTAS V ON C.ID = V.ID_CLIENTE
GROUP BY C.NOMBRE
ORDER BY TOTAL_VENDIDO DESC;
```

GROUP BY agrupa las filas por cliente y SUM() suma sus montos.

NOMBRE	TOTAL_VENDIDO
Juan	250000
Pedro	200000
Maria	50000

c) Cliente con la venta mas baja

```
SELECT C.NOMBRE, V.MONTO
FROM   CLIENTES C
INNER JOIN VENTAS V ON C.ID = V.ID_CLIENTE
WHERE  V.MONTO = (SELECT MIN(MONTO) FROM VENTAS);
```

Se usa una subconsulta con MIN: es la forma recomendada porque soporta empates (si dos clientes tuvieran el monto minimo, los mostraria a ambos).

NOMBRE	MONTO
Maria	50000

Resultado: Maria - 50000 (la venta individual mas baja).

Problema 2 - Data Warehouse y SQL Analitico

Ejemplo propio en ESQUEMA ESTRELLA: una tabla de hechos FACT_VENTAS con dos dimensiones, DIM_TIEMPO y DIM_PRODUCTO. La tabla de hechos guarda las medidas numericas (monto, cantidad) y las dimensiones aportan el contexto (tiempo, producto) por el que se agrupa.

Modelo dimensional (resumen)

DIM_TIEMPO	id_tiempo, fecha, mes, trimestre, anio
DIM_PRODUCTO	id_producto, nombre_producto, categoria, marca
FACT_VENTAS	id_venta, id_tiempo, id_producto, cantidad, monto

a) Ventas totales por anio

```
SELECT T.ANIO, SUM(F.MONTO) AS VENTAS_TOTALES
FROM   FACT_VENTAS F
INNER JOIN DIM_TIEMPO T ON F.ID_TIEMPO = T.ID_TIEMPO
GROUP BY T.ANIO
ORDER BY T.ANIO;
```

ANIO	VENTAS_TOTALES
2023	1775000
2024	1170000

b) Ventas por categoria

```
SELECT P.CATEGORIA, SUM(F.MONTO) AS VENTAS_CATEGORIA
FROM   FACT_VENTAS F
INNER JOIN DIM_PRODUCTO P ON F.ID_PRODUCTO = P.ID_PRODUCTO
GROUP BY P.CATEGORIA
ORDER BY VENTAS_CATEGORIA DESC;
```

CATEGORIA	VENTAS_CATEGORIA
Tecnologia	2525000
Muebles	270000
Alimentos	150000

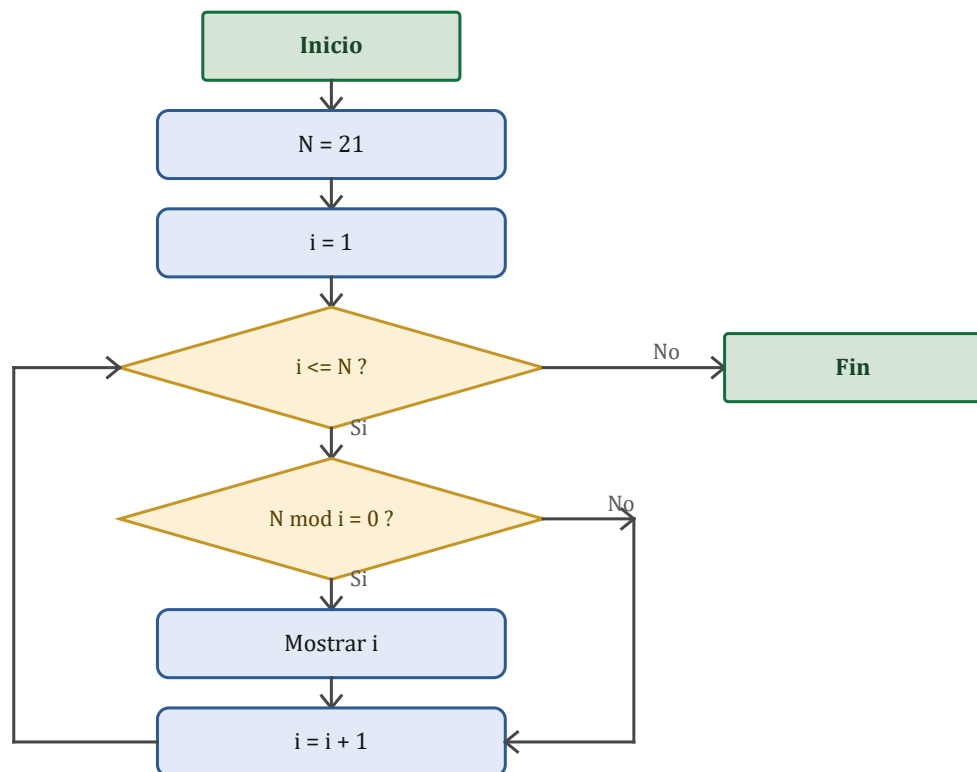
Problema 3 - Divisores de 21

Los divisores de 21 son 1, 3, 7 y 21 (porque $21 = 3 \times 7$). Se recorre del 1 al 21 y se muestran los números que dividen a 21 sin dejar resto ($21 \bmod i = 0$).

Pseudocodigo

```
Algoritmo Divisores_de_21
  Definir N, i Como Entero
  N <- 21
  Para i <- 1 Hasta N Con Paso 1 Hacer
    Si (N mod i = 0) Entonces
      Escribir i, " es divisor de ", N
    FinSi
  FinPara
FinAlgoritmo
```

Diagrama de flujo



Algoritmo en SQL (CTE recursivo)

```
WITH NUMEROS AS (
  SELECT 1 AS I
  UNION ALL
  SELECT I + 1 FROM NUMEROS WHERE I < 21
)
SELECT I AS DIVISOR FROM NUMEROS WHERE 21 % I = 0; -- en T-SQL el modulo es %
```

Resultado: 1, 3, 7, 21.

Funcion con el algoritmo (funcion en linea con valores de tabla)

```
CREATE FUNCTION FN_DIVISORES (@N INT)
RETURNS TABLE
AS RETURN
(
    WITH NUMEROS AS (
        SELECT 1 AS I
        UNION ALL
        SELECT I + 1 FROM NUMEROS WHERE I < @N
    )
    SELECT I AS DIVISOR FROM NUMEROS WHERE @N % I = 0
);

SELECT * FROM FN_DIVISORES(21);
```

El script tambien incluye el procedimiento SP_DIVISORES, que implementa el pseudocodigo paso a paso con un bucle WHILE y un IF (ver anexo).

Problema 4 - Esquema Estrella vs Copo de Nieve

Ejemplo propio y distinto al de clases: una plataforma de STREAMING DE MUSICA donde se mide la reproduccion de canciones. La medida vive en la tabla de hechos FACT_REPRODUCCIONES.

Esquema Estrella

Las dimensiones son tablas unicas y anchas (desnormalizadas). Por ejemplo, DIM_CANCION guarda en la misma fila el titulo, el artista, el album y el genero (datos que se repiten en cada cancion).

Esquema Copo de Nieve

La dimension cancion se normaliza en varias tablas relacionadas: DIM_CANCION -> DIM_ALBUM -> DIM_ARTISTA, y DIM_GENERO aparte. Asi no hay datos repetidos.

Ventajas y desventajas

Aspecto	Estrella	Copo de nieve
Normalizacion	Desnormalizado (datos repetidos)	Normalizado (sin redundancia)
N de JOINS	Pocos: consultas mas rapidas	Muchos: consultas mas lentas
Almacenamiento	Usa mas espacio	Usa menos espacio
Mantenimiento	Mas dificil (datos repetidos)	Mas facil (un solo lugar)
Integridad	Mayor riesgo de inconsistencia	Mayor consistencia
Ideal para	Reportes / BI rapidos	Dimensiones grandes con jerarquias

En resumen: la estrella prioriza la velocidad de consulta; el copo de nieve prioriza el ahorro de espacio y la integridad. En BI suele preferirse la estrella por su rendimiento.

Consultas de alta complejidad

1) Top 3 artistas por tiempo total escuchado (recorre toda la jerarquia del copo de nieve)

```
SELECT TOP 3 A.NOMBRE_ARTISTA,
      COUNT(*) AS TOTAL_REPRODUCCIONES,
      SUM(F.DURACION_ESCUCHADA) AS SEGUNDOS_TOTALES,
      ROUND(AVG(F.DURACION_ESCUCHADA*1.0),1) AS PROMEDIO_SEGUNDOS
FROM   FACT_REPRODUCCIONES F
JOIN   DIM_CANCION C ON F.ID_CANCION = C.ID_CANCION
JOIN   DIM_ALBUM AL ON C.ID_ALBUM = AL.ID_ALBUM
JOIN   DIM_ARTISTA A ON AL.ID_ARTISTA = A.ID_ARTISTA
GROUP BY A.NOMBRE_ARTISTA
ORDER BY SEGUNDOS_TOTALES DESC;
```

2) Por genero: reproducciones, cancion mas larga, mas corta y promedio (MAX, MIN, AVG, HAVING)

```
SELECT G.NOMBRE_GENERO, COUNT(*) AS REPRODUCCIONES,
      MAX(C.DURACION_SEGUNDOS) AS CANCION_MAS_LARGA,
      MIN(C.DURACION_SEGUNDOS) AS CANCION_MAS_CORTA,
      ROUND(AVG(C.DURACION_SEGUNDOS*1.0),1) AS DURACION_PROMEDIO
FROM   FACT_REPRODUCCIONES F
JOIN   DIM_CANCION C ON F.ID_CANCION = C.ID_CANCION
JOIN   DIM_GENERO G ON C.ID_GENERO = G.ID_GENERO
GROUP BY G.NOMBRE_GENERO
HAVING COUNT(*) > 0
ORDER BY REPRODUCCIONES DESC;
```

3) SQL analitico: participacion (%) de cada plan dentro de cada pais (funcion de ventana)

```
SELECT U.PAIS, U.PLAN_USR, COUNT(*) AS REPRODUCCIONES,  
       SUM(COUNT(*)) OVER (PARTITION BY U.PAIS) AS TOTAL_PAIS,  
       ROUND(100.0*COUNT(*) /  
             SUM(COUNT(*)) OVER (PARTITION BY U.PAIS),1) AS PCT_DENTRO_PAIS  
FROM   FACT_REPRODUCCIONES F  
JOIN   DIM_USUARIO U ON F.ID_USUARIO = U.ID_USUARIO  
GROUP BY U.PAIS, U.PLAN_USR  
ORDER BY U.PAIS, REPRODUCCIONES DESC;
```

La ultima consulta combina agregacion (GROUP BY) con una funcion de ventana SUM() OVER (PARTITION BY ...), calculando el total por pais sin colapsar las filas y obteniendo el porcentaje de cada plan dentro de su pais.

Conclusion

El examen integra las consultas SQL fundamentales (JOIN, GROUP BY, subconsultas y agregaciones) con conceptos de Data Warehouse (modelos dimensionales estrella y copo de nieve) y SQL analitico (funciones de ventana). El Problema 3 ademas conecta el pensamiento algoritmico (pseudocodigo y diagrama de flujo) con su implementacion en SQL mediante un CTE recursivo, un procedimiento con bucle y una funcion. Todo el codigo esta escrito en T-SQL (SQL Server) y su sintaxis fue validada.

Anexo - Script SQL completo

Codigo fuente del entregable Examen_Final_Pablo_Villegas.sql:

```

/*****
EXAMEN FINAL: SQL
Facultad de Ingenieria y Negocios - Area de Estadistica (UDLA)
Estudiante: Pablo Villegas
Fecha: 15 de junio de 2026
Dialecto: T-SQL (SQL Server)
*****/

/*****
PROBLEMA 1 - PARTE IV: SQL (JOIN Y AGREGACIONES)
Tablas CLIENTES y VENTAS.
*****/

-----
-- creacion de tablas
-----
CREATE TABLE CLIENTES (
    ID      INT PRIMARY KEY,
    NOMBRE  VARCHAR(50),
    CIUDAD  VARCHAR(50)
);

CREATE TABLE VENTAS (
    ID_VENTA INT PRIMARY KEY,
    ID_CLIENTE INT,
    MONTO     INT,
    FOREIGN KEY (ID_CLIENTE) REFERENCES CLIENTES(ID)
);

-----
-- insertar registros (datos del enunciado)
-----
INSERT INTO CLIENTES (ID, NOMBRE, CIUDAD) VALUES
(1, 'JUAN', 'CONCEPCION'),
(2, 'MARIA', 'LOS ANGELES'),
(3, 'PEDRO', 'CONCEPCION');

INSERT INTO VENTAS (ID_VENTA, ID_CLIENTE, MONTO) VALUES
(1, 1, 100000),
(2, 1, 150000),
(3, 2, 50000),
(4, 3, 200000);

-----
-- a) Mostrar NOMBRE, CIUDAD y MONTO utilizando JOIN
-- El INNER JOIN une cada venta con su cliente por C.ID = V.ID_CLIENTE.
-----
SELECT  C.NOMBRE,
        C.CIUDAD,
        V.MONTO
FROM    CLIENTES C
INNER JOIN VENTAS V ON C.ID = V.ID_CLIENTE;

-----
-- b) Total vendido por cliente (SUM + GROUP BY)
-- Resultado: Juan 250000, Maria 50000, Pedro 200000.
-----
SELECT  C.NOMBRE,
        SUM(V.MONTO) AS TOTAL_VENDIDO
FROM    CLIENTES C
INNER JOIN VENTAS V ON C.ID = V.ID_CLIENTE
GROUP BY C.NOMBRE
ORDER BY TOTAL_VENDIDO DESC;

-----
-- c) Cliente con la venta MAS BAJA

```

```
-- Con subconsulta MIN: soporta empates y es la forma recomendada.
-- Resultado: Maria - 50000.
```

```
-----
SELECT  C.NOMBRE,
        V.MONTO
FROM    CLIENTES C
INNER JOIN VENTAS V ON C.ID = V.ID_CLIENTE
WHERE   V.MONTO = (SELECT MIN(MONTO) FROM VENTAS);
```

```
-- Forma alternativa en SQL Server (ordenando y tomando el primero):
SELECT TOP 1
        C.NOMBRE,
        V.MONTO
FROM    CLIENTES C
INNER JOIN VENTAS V ON C.ID = V.ID_CLIENTE
ORDER BY V.MONTO ASC;
```

```
-----
/*****
PROBLEMA 2 - PARTE V: DATA WAREHOUSE Y SQL ANALITICO
Modelo en ESQUEMA ESTRELLA: una tabla de hechos FACT_VENTAS con dos
dimensiones (DIM_TIEMPO y DIM_PRODUCTO). Ejemplo creado por el estudiante.
*****/
```

```
-----
-- creacion del modelo dimensional
-----
```

```
CREATE TABLE DIM_TIEMPO (
    ID_TIEMPO INT PRIMARY KEY,
    FECHA     DATE,
    MES       INT,
    TRIMESTRE INT,
    ANIO      INT
);
```

```
CREATE TABLE DIM_PRODUCTO (
    ID_PRODUCTO INT PRIMARY KEY,
    NOMBRE_PRODUCTO VARCHAR(100),
    CATEGORIA      VARCHAR(50),
    MARCA          VARCHAR(50)
);
```

```
CREATE TABLE FACT_VENTAS (
    ID_VENTA INT PRIMARY KEY,
    ID_TIEMPO INT,
    ID_PRODUCTO INT,
    CANTIDAD INT,
    MONTO INT,
    FOREIGN KEY (ID_TIEMPO) REFERENCES DIM_TIEMPO(ID_TIEMPO),
    FOREIGN KEY (ID_PRODUCTO) REFERENCES DIM_PRODUCTO(ID_PRODUCTO)
);
```

```
-----
-- datos de ejemplo
-----
```

```
INSERT INTO DIM_TIEMPO (ID_TIEMPO, FECHA, MES, TRIMESTRE, ANIO) VALUES
(1, '2023-01-15', 1, 1, 2023),
(2, '2023-06-20', 6, 2, 2023),
(3, '2024-03-10', 3, 1, 2024),
(4, '2024-11-05', 11, 4, 2024);
```

```
INSERT INTO DIM_PRODUCTO (ID_PRODUCTO, NOMBRE_PRODUCTO, CATEGORIA, MARCA) VALUES
(1, 'LAPTOP PRO', 'TECNOLOGIA', 'DELL'),
(2, 'MOUSE GAMER', 'TECNOLOGIA', 'LOGITECH'),
(3, 'CAFE PREMIUM', 'ALIMENTOS', 'JUAN VALDEZ'),
(4, 'SILLA ERGONOMICA', 'MUEBLES', 'IKEA');
```

```
INSERT INTO FACT_VENTAS (ID_VENTA, ID_TIEMPO, ID_PRODUCTO, CANTIDAD, MONTO) VALUES
(1, 1, 1, 2, 1600000),
(2, 1, 3, 10, 50000),
(3, 2, 2, 5, 125000),
(4, 3, 1, 1, 800000),
(5, 3, 4, 3, 270000);
```

```
(6, 4, 3, 20, 100000);
```

```
-----
-- a) Ventas totales por año
-- Resultado: 2023 -> 1775000 ; 2024 -> 1170000.
```

```
-----
SELECT T.ANIO,
       SUM(F.MONTO) AS VENTAS_TOTALES
FROM   FACT_VENTAS F
INNER JOIN DIM_TIEMPO T ON F.ID_TIEMPO = T.ID_TIEMPO
GROUP BY T.ANIO
ORDER BY T.ANIO;
```

```
-----
-- b) Ventas por categoría
-- Resultado: Tecnología 2525000, Muebles 270000, Alimentos 150000.
```

```
-----
SELECT P.CATEGORIA,
       SUM(F.MONTO) AS VENTAS_CATEGORIA
FROM   FACT_VENTAS F
INNER JOIN DIM_PRODUCTO P ON F.ID_PRODUCTO = P.ID_PRODUCTO
GROUP BY P.CATEGORIA
ORDER BY VENTAS_CATEGORIA DESC;
```

```
-----
/*****
```

```
PROBLEMA 3 - DIVISORES DE 21
Los divisores de 21 son: 1, 3, 7 y 21 (porque 21 = 3 x 7).
```

```
PSEUDOCODIGO
```

```
-----
```

```
Algoritmo Divisores_de_21
  Definir N, i Como Entero
  N <- 21
  Para i <- 1 Hasta N Con Paso 1 Hacer
    Si (N mod i = 0) Entonces
      Escribir i, " es divisor de ", N
    FinSi
  FinPara
FinAlgoritmo
```

```
DIAGRAMA DE FLUJO (descripcion textual)
```

```
-----
```

```
(Inicio) -> [N = 21] -> [i = 1] -> <i <= N?>
  <i <= N?> -- No --> (Fin)
  <i <= N?> -- Si --> <N mod i = 0?>
    <N mod i = 0?> -- Si --> [Mostrar i] -> [i = i + 1] -> (vuelve a <i <= N?>)
    <N mod i = 0?> -- No --> [i = i + 1] -> (vuelve a <i <= N?>)
```

```
-----
/
```

```
-----
-- Algoritmo en SQL: CTE recursivo que genera 1..21 y filtra los divisores.
-- En T-SQL el modulo se escribe con el operador % (no se usa la palabra RECURSIVE).
```

```
-----
WITH NUMEROS AS (
  SELECT 1 AS I
  UNION ALL
  SELECT I + 1 FROM NUMEROS WHERE I < 21
)
SELECT I AS DIVISOR
FROM   NUMEROS
WHERE  21 % I = 0;
```

```
-----
-- Implementacion paso a paso del pseudocodigo: procedimiento con bucle WHILE.
```

```
-----
CREATE PROCEDURE SP_DIVISORES @N INT
AS
BEGIN
  DECLARE @I INT = 1;
  DECLARE @RESULTADO TABLE (DIVISOR INT);

  WHILE @I <= @N
```

```

BEGIN
    IF @N % @I = 0
        INSERT INTO @RESULTADO (DIVISOR) VALUES (@I);
    SET @I = @I + 1;
END

SELECT * FROM @RESULTADO;
END;

EXEC SP_DIVISORES 21;

-----
-- Funcion con el algoritmo del enunciado (funcion en linea con valores de tabla).
-- Devuelve la tabla de divisores de cualquier numero @N.
-----
CREATE FUNCTION FN_DIVISORES (@N INT)
RETURNS TABLE
AS
RETURN
(
    WITH NUMEROS AS (
        SELECT 1 AS I
        UNION ALL
        SELECT I + 1 FROM NUMEROS WHERE I < @N
    )
    SELECT I AS DIVISOR
    FROM NUMEROS
    WHERE @N % I = 0
);

-- Uso de la funcion:
SELECT * FROM FN_DIVISORES(21);

```

```

/*****
PROBLEMA 4 - ESQUEMA ESTRELLA VS COPO DE NIEVE
Ejemplo propio (distinto al de clases): plataforma de STREAMING DE MUSICA.
Se mide la REPRODUCCION de canciones.

```

VENTAJAS / DESVENTAJAS

```

-----
ESTRELLA (dimensiones desnormalizadas, "anchas"):
+ Menos JOINS -> consultas mas rapidas y simples (ideal para BI/reportes).
+ Modelo facil de entender.
- Datos repetidos -> usa mas espacio y hay riesgo de inconsistencias.
- Mantenimiento mas costoso (hay que actualizar valores repetidos).

COPO DE NIEVE (dimensiones normalizadas en varias tablas):
+ Sin redundancia -> menos espacio y mayor integridad de datos.
+ Mantenimiento mas facil (un cambio en un solo lugar).
- Muchos JOINS -> consultas mas lentas y complejas.

```

En resumen: la ESTRELLA prioriza la velocidad de consulta; el COPO DE NIEVE prioriza el ahorro de espacio y la integridad. En BI suele preferirse la estrella.

```

*****/

```

```

-----
-- ESQUEMA ESTRELLA: dimensiones unicas y "anchas" (desnormalizadas)
-----

```

```

CREATE TABLE DIM_USUARIO (
    ID_USUARIO INT PRIMARY KEY,
    NOMBRE     VARCHAR(100),
    PAIS       VARCHAR(50),
    PLAN_USR   VARCHAR(20)      -- 'FREE' / 'PREMIUM'
);

```

```

-- Version ESTRELLA de la cancion: artista, album y genero van en la misma tabla.

```

```

CREATE TABLE DIM_CANCION_ESTRELLA (
    ID_CANCION      INT PRIMARY KEY,
    TITULO          VARCHAR(150),
    ARTISTA         VARCHAR(100),  -- datos repetidos por cada cancion
    ALBUM           VARCHAR(100),
    GENERO          VARCHAR(50),

```

```

    DURACION_SEGUNDOS INT
);

-----
-- ESQUEMA COPO DE NIEVE: la dimension cancion se NORMALIZA en varias tablas
-- (cancion -> album -> artista, y genero aparte)
-----
CREATE TABLE DIM_GENERO (
    ID_GENERO      INT PRIMARY KEY,
    NOMBRE_GENERO  VARCHAR(50)
);

CREATE TABLE DIM_ARTISTA (
    ID_ARTISTA     INT PRIMARY KEY,
    NOMBRE_ARTISTA VARCHAR(100),
    PAIS_ORIGEN    VARCHAR(50)
);

CREATE TABLE DIM_ALBUM (
    ID_ALBUM        INT PRIMARY KEY,
    NOMBRE_ALBUM     VARCHAR(100),
    ID_ARTISTA       INT,
    ANIO_LANZAMIENTO INT,
    FOREIGN KEY (ID_ARTISTA) REFERENCES DIM_ARTISTA(ID_ARTISTA)
);

CREATE TABLE DIM_CANCION (
    ID_CANCION      INT PRIMARY KEY,
    TITULO           VARCHAR(150),
    ID_ALBUM         INT,
    ID_GENERO        INT,
    DURACION_SEGUNDOS INT,
    FOREIGN KEY (ID_ALBUM) REFERENCES DIM_ALBUM(ID_ALBUM),
    FOREIGN KEY (ID_GENERO) REFERENCES DIM_GENERO(ID_GENERO)
);

-- Tabla de hechos (compartida): apunta a la dimension cancion del copo de nieve.
CREATE TABLE FACT_REPRODUCCIONES (
    ID_REPRODUCCION INT PRIMARY KEY,
    ID_USUARIO       INT,
    ID_CANCION       INT,
    DURACION_ESCUCHADA INT,      -- segundos efectivamente escuchados
    DISPOSITIVO       VARCHAR(30),
    FOREIGN KEY (ID_USUARIO) REFERENCES DIM_USUARIO(ID_USUARIO),
    FOREIGN KEY (ID_CANCION) REFERENCES DIM_CANCION(ID_CANCION)
);

-----
-- datos de ejemplo (copo de nieve)
-----
INSERT INTO DIM_USUARIO (ID_USUARIO, NOMBRE, PAIS, PLAN_USR) VALUES
(1, 'CAMILA', 'CHILE', 'PREMIUM'),
(2, 'DIEGO', 'CHILE', 'FREE'),
(3, 'LUCIA', 'ARGENTINA', 'PREMIUM'),
(4, 'MATEO', 'ARGENTINA', 'FREE');

INSERT INTO DIM_GENERO (ID_GENERO, NOMBRE_GENERO) VALUES
(1, 'ROCK'),
(2, 'POP'),
(3, 'JAZZ');

INSERT INTO DIM_ARTISTA (ID_ARTISTA, NOMBRE_ARTISTA, PAIS_ORIGEN) VALUES
(1, 'LOS ANDES', 'CHILE'),
(2, 'LUNA AZUL', 'ARGENTINA'),
(3, 'TRIO CENTRAL', 'CHILE');

INSERT INTO DIM_ALBUM (ID_ALBUM, NOMBRE_ALBUM, ID_ARTISTA, ANIO_LANZAMIENTO) VALUES
(1, 'CUMBRE', 1, 2022),
(2, 'NOCHE', 2, 2023),
(3, 'IMPROVISTO', 3, 2021);

INSERT INTO DIM_CANCION (ID_CANCION, TITULO, ID_ALBUM, ID_GENERO, DURACION_SEGUNDOS) VALUES
(1, 'MONTANA', 1, 1, 210),

```

```
(2, 'SENDERO', 1, 1, 185),
(3, 'ESTRELLAS', 2, 2, 240),
(4, 'AMANECER', 2, 2, 200),
(5, 'IMPROVISO', 3, 3, 320);
```

```
INSERT INTO FACT_REPRODUCCIONES (ID_REPRODUCCION, ID_USUARIO, ID_CANCION, DURACION_ESCUCHADA, DISPOSITIVO) VALUES
(1, 1, 1, 210, 'MOVIL'),
(2, 1, 3, 240, 'WEB'),
(3, 2, 1, 120, 'MOVIL'),
(4, 2, 5, 300, 'TV'),
(5, 3, 3, 240, 'MOVIL'),
(6, 3, 4, 200, 'WEB'),
(7, 4, 5, 150, 'MOVIL'),
(8, 4, 2, 185, 'WEB');
```

```
/******
CONSULTAS DE ALTA COMPLEJIDAD (SUM, AVG, MAX, MIN, HAVING y funciones de ventana)
******/
```

```
-- 1) Top 3 artistas por tiempo total escuchado
-- Recorre toda la jerarquía del copo de nieve: cancion -> album -> artista.
```

```
-----
SELECT TOP 3
    A.NOMBRE_ARTISTA,
    COUNT(*) AS TOTAL_REPRODUCCIONES,
    SUM(F.DURACION_ESCUCHADA) AS SEGUNDOS_TOTALES,
    ROUND(AVG(F.DURACION_ESCUCHADA * 1.0), 1) AS PROMEDIO_SEGUNDOS
FROM FACT_REPRODUCCIONES F
INNER JOIN DIM_CANCION C ON F.ID_CANCION = C.ID_CANCION
INNER JOIN DIM_ALBUM AL ON C.ID_ALBUM = AL.ID_ALBUM
INNER JOIN DIM_ARTISTA A ON AL.ID_ARTISTA = A.ID_ARTISTA
GROUP BY A.NOMBRE_ARTISTA
ORDER BY SEGUNDOS_TOTALES DESC;
```

```
-----
-- 2) Por genero: reproducciones, cancion mas larga, mas corta y duracion promedio
-- (COUNT, MAX, MIN, AVG con GROUP BY y HAVING)
```

```
-----
SELECT G.NOMBRE_GENERO,
    COUNT(*) AS REPRODUCCIONES,
    MAX(C.DURACION_SEGUNDOS) AS CANCION_MAS_LARGA,
    MIN(C.DURACION_SEGUNDOS) AS CANCION_MAS_CORTA,
    ROUND(AVG(C.DURACION_SEGUNDOS * 1.0), 1) AS DURACION_PROMEDIO
FROM FACT_REPRODUCCIONES F
INNER JOIN DIM_CANCION C ON F.ID_CANCION = C.ID_CANCION
INNER JOIN DIM_GENERO G ON C.ID_GENERO = G.ID_GENERO
GROUP BY G.NOMBRE_GENERO
HAVING COUNT(*) > 0
ORDER BY REPRODUCCIONES DESC;
```

```
-----
-- 3) SQL ANALITICO: participacion (%) de cada PLAN dentro de cada PAIS
-- Combina agregacion (GROUP BY) con funcion de ventana SUM() OVER (PARTITION BY).
```

```
-----
SELECT U.PAIS,
    U.PLAN_USR,
    COUNT(*) AS REPRODUCCIONES,
    SUM(COUNT(*)) OVER (PARTITION BY U.PAIS) AS TOTAL_PAIS,
    ROUND(100.0 * COUNT(*) /
        SUM(COUNT(*)) OVER (PARTITION BY U.PAIS), 1) AS PCT_DENTRO_PAIS
FROM FACT_REPRODUCCIONES F
INNER JOIN DIM_USUARIO U ON F.ID_USUARIO = U.ID_USUARIO
GROUP BY U.PAIS, U.PLAN_USR
ORDER BY U.PAIS, REPRODUCCIONES DESC;
```